

Comparison of the two approaches – WITH PANDAS & WITHOUT PANDAS

Results agree?

Yes. Both scripts report the same numbers for `spend`:

Statistic	Without Pandas	Pandas
Count	246,745	246,745
Mean	1,061.79	1,061.79
Min	49.50	49.50
Max	474,999.50	474,999.50
Median	49.50	49.50
stdev	(not computed)	4,992.56

They matched because both scripts convert the range-encoded columns (`spend`, `impressions`, `estimated\_audience\_size`) the same way before computing statistics.

Where pure Python forced decisions, Pandas made silently?

1. **Type detection:** In the pure Python script, numeric vs. categorical is decided by a fixed list of known numeric columns (`RANGE\_COLUMNS` and the 0/1 flag columns). Pandas infers dtypes automatically from `read\_csv`, but by default treats `spend`, `impressions`, and `estimated\_audience\_size` as text, since they are stored as `{'lower\_bound': ..., 'upper\_bound': ...}` strings. Without fixing this in both scripts, those three columns would be missing from the numeric summary entirely.
2. **Missing values:** The pure Python script treats an empty string as missing explicitly. Pandas' `read\_csv`/`isna()` does this automatically using its own default rules.
3. **Standard deviation with one value:** The pure Python script returns `0` if a column has only one usable value, to avoid a crash. Pandas' `.std()` returns `NaN` in the same situation.

Writing the pure Python version showed that Pandas alone would not have?

Writing the manual parser for `spend`/`impressions`/`estimated\_audience\_size` is what surfaced the fact that these columns are not plain numbers. Running `df.describe()` in Pandas without that fix would have simply left those three columns out of the numeric summary, with no error or warning.